

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении практической работы №3
по дисциплине «Функциональное программирование»
Вариант №17.

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Моторин Д.Е.

«_____» _____ 2025г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Постановка задачи	4
2 Расширение модели финансовых транзакций	5
2.1 Работа с типом Budget	5
2.2 Functor для типа TransactionValidator a	6
2.3 Applicative для типа TransactionValidator a	7
3 Проверка корректности	8
Выводы	10
Список использованных источников	11
Приложение 1 «Исходный код программы»	12

Введение

Финансовая транзакция - это запись о движении денежных средств между сторонами в результате любой сделки или операции с деньгами или финансовыми активами.

В каждой транзакции должны быть: уникальный идентификатор, сумма, тип операции и ее описание. Нередко добавляют точную дату и время, валюту, источника и получателя денежных средств.

Основные типы операций связаны с направлением денежных потоков:

1. Доход - поступление денег на счет.
2. Расход - убытие средств со счета.
3. Перевод - перемещение денег между своими счетами.
4. Депозит - накопление средств.

Транзакции подчиняются некоторым бизнес-правилам:

1. Данные должны быть целостны: сумма всегда положительная, все обязательные поля заполнены.
2. Данные должны быть валидны: счета, валюты, даты, персональная информация корректная и подтвержденная.
3. Завершенные транзакции не могут быть изменены.
4. Во время выполнения транзакции параллельно ей не может выполняться другая транзакция, использующая те же счета. В случае сбоя транзакции движение всех денежных средств отменяется.
5. Каждой транзакции присваивается только одна категория.

Модели финансовых транзакций используются для учета денежных потоков, налоговой отчетности, анализа и планирования бюджета.

1 Постановка задачи

Цель: расширение разработанной во второй практической работе модели финансовой транзакций.

Задачи:

1. Определить тип бюджета категории Budget.
2. Реализовать представителей классов Semifroup и Monoid для типа Budget.
3. Написать функцию concatBudgets.
4. Определить тип валидатора транзакций TransactionValidator a.
5. Реализовать представителя класса Functor и Applicative для типа TransactionValidator.
6. Написать функцию simpleValidator.
7. Написать функцию combineValidations.

2 Расширение модели финансовых транзакций

2.1 Работа с типом Budget

Тип Budget определен как тип-произведение, который объединяет несколько полей в одну структуру: запланированную сумму типа Double, потраченную сумму типа Double, лимит предупреждения типа Double. Тип получает возможность сравнения и вывода на печать.

Листинг 1. Определение типа Budget

```
1 data Budget = Budget{  
2   planSum :: Double  
3   , spentSum :: Double  
4   , limitWarning :: Double  
5 } deriving (Eq, Show)
```

В листинге 2 приведен представитель класса Semigroup для типа Budget. Операция ($\langle \rangle$) объединяет два бюджета: суммирует запланированные и потраченные суммы, выбирает минимальный лимит предупреждения.

Листинг 2. Представитель класса Semigroup для типа Budget

```
1 instance Semigroup Budget where  
2 (<>) (Budget p1 s1 l1) (Budget p2 s2 l2) = Budget (p1+p2) (s1+s2) (min  
   l1 l2)
```

В листинге 3 приведен представитель класса Monoid для типа Budget. В качестве нейтрального элемента mempty выбран бюджет с нулевыми запланированными и потраченными суммами и бесконечно большим лимитом предупреждения. При сложении с бюджетом, где суммы равны нулю, исходные суммы не меняются. Для операции min, применяемой при сравнении лимитов, бесконечность является наибольшим возможным числом. В качестве mappend выбрана бинарная операция ($\langle \rangle$), реализованная в представителе Semigroup.

Листинг 3. Представитель класса Monoid для типа Budget

```
1 instance Monoid Budget where  
2 mempty = Budget 0 0 (1/0)  
3 mappend = (<>)
```

Функция concatBudgets объединяет бюджеты подкатегорий в общий бюджет категории.

Листинг 4. Функция concatBudgets

```
1 concatBudgets :: [Budget] -> Budget
```

```
2 concatBudgets = mconcat
```

Для ее реализации необходима стандартная функция `mconcat`, которая объединяет все бюджеты из списка, начиная с нейтрального элемента.

2.2 Functor для типа TransactionValidator a

Тип `TransactionValidator` определен как валидатор транзакций с дополнительными данными, где `Budget` - текущий бюджет, `Bool` - прошла ли транзакция валидацию, `a` - дополнительная информация.

Листинг 5. Тип `TransactionValidator a`

```
1 newtype TransactionValidator a = TransactionValidator {
2   runValidation :: Transaction -> Budget -> (Bool, a)
3 }
```

В листинге 6 приведен представитель класса `Functor` для типа `TransactionValidator`. Функция `fmap` преобразует дополнительную информацию валидации, при этом булево значение результата прохождения валидации остается неизменным.

Листинг 6. Представитель класса `Functor` типа `TransactionValidator a`

```
1 instance Functor TransactionValidator where
2   fmap f (TransactionValidator val) = TransactionValidator (\ t b ->
3     let (res, info) = val t b
4     in (res, f info))
```

Функция `simpleValidator` модифицирует результаты валидации.

Листинг 7. Функция `simpleValidator`

```
1 simpleValidator :: TransactionValidator String
2 simpleValidator = TransactionValidator $ \t b ->
3 let remaining = getRemaining b
4 transactionSum = summa t
5 isValid = transactionSum <= remaining
6 message = if isValid then "Transaction approved. Remaining after: " ++
7   show (remaining - transactionSum)
8 else "Amount " ++ show transactionSum ++ " exceeds remaining budget " ++
9   show remaining
10 in (isValid, message)
```

Она изменяет дополнительную информацию в зависимости от входных данных транзакции. Если сумма транзакции превышает величину оставшегося бюджета, в сообщение записывается предупреждение «Amount exceeds remaining budget». Иначе записывается подтверждение «Transaction approved».

2.3 Applicative для типа TransactionValidator a

В листинге 8 приведен представитель класса Applicative для типа TransactionValidator a. Операция ($<*>$) комбинирует два валидатора; результат проходит только если оба валидатора прошли.

Функция pure создает нейтральный контекст, который не должен влиять на вычисления. В качестве семантики pure создается валидатор, в котором булево значение всегда True и значение передается без изменений. Параметры транзакции и бюджета игнорируются.

Листинг 8. Представитель класса Applicative типа TransactionValidator a

```
1 instance Applicative TransactionValidator where
2 pure x = TransactionValidator $ \_ _ -> (True, x)
3 (TransactionValidator f) <*> (TransactionValidator s) =
4   TransactionValidator $ \t b ->
5   let (fRes, fFunc) = f t b
6   (vRes, vVal) = s t b
7   in (fRes && vRes, fFunc vVal)
```

Функция combineValidations комбинирует несколько условий валидации. Функция принимает на вход список экземпляров типа TransactionValidator a и возвращает валидатор, который производит список результатов.

Листинг 9. Функция combineValidations

```
1 combineValidations :: [TransactionValidator String] ->
2   TransactionValidator [String]
3 combineValidations [] = pure []
4 combineValidations (v:vs) = (:) <$> v <*> combineValidations vs
```

Если список пустой - в результате возвращается нейтральный элемент pure. Иначе список разделяется на хвост и голову, операция $<*>$ рекурсивно применяется для хвоста.

3 Проверка корректности

Для тестирования корректности функции `concatBudgets` созданы три бюджета с различными планируемыми, потраченными суммами и лимитом предупреждения. В результате работы функции выведен общий бюджет категории (рисунок 1).

```
let food = Budget 2000 1600 100
    transport = Budget 1000 1000 200
    dorm = Budget 3000 2700 300
    total = concatBudgets [food, transport, dorm]

Summary budget: Budget {planSum = 6000.0, spentSum = 5300.0, limitWarning = 100.0}
```

Рис. 1. Проверка работы функции `concatBudgets`

Для проверки корректности функции модификации результатов валидации создан тестовый бюджет и две транзакции. Выводимое сообщение зависит от того, превышает ли оставшийся бюджет сумму транзакции. На рисунке 2 показаны результаты работы функции в обоих случаях.

```
Transaction: 200.5 for Groceries
Budget: 3000.0 in all
Result: True
Message: Transaction approved. Remaining after: 2799.5

Transaction: 4000.0 for Meat
Budget: 3000.0 in all
Result: False
Message: Amount 4000.0 exceeds remaining budget 3000.0
```

Рис. 2. Проверка работы функции `simpleValidator`

Для проверки функции комбинации нескольких валидаторов созданы три валидатора: на проверку суммы (сумма не превышает остатка бюджета), категории (разрешены только типы категорий `Expense` и `Income`) и лимита (остаток бюджета больше, чем лимит предупреждения). Также созданы две транзакции. Результаты работы программы приведены на рисунке 3.


```
Result: True
Amount OK: 800.0 <= 3000.0
Category OK: Expense
Above warning limit: 2200.0 >= 500.0

Result: False
Amount OK: 1900.0 <= 3000.0
Invalid category: Transfer
Above warning limit: 1100.0 >= 500.0
```

Рис. 3. Проверка работы функции combineValidations

Тестирование подтвердило, что функция объединяет результаты нескольких валидаторов и разрешает прохождение транзакции только тогда, когда все условия выполнены.

Выводы

В ходе работы успешно расширена модель финансовых транзакций. Для этого определен тип `Budget`, реализованы представители класса `Semigroup` и `Monoid`, а также функция `concatBudgets` для объединения бюджетов подкатегорий в общий бюджет. Определен тип `TransactionValidator` а, реализованы представители классов `Functor` и `Applicative`. Написаны функция `simpleValidator`, модифицирующая результаты валидации, а также функция `combineValidations`, комбинирующая несколько условий валидации. Тестирование программы показало, что все функции работают корректно и обрабатывают частные случаи.

Список литературы

- [1] Курт У. «Программируй на Haskell». М.: Издательство, 2019. 320 стр.
- [2] Липовача М. «Изучай Haskell во имя добра». М.: ДМК Пресс, 2012. 490 стр.

Приложение 1 «Исходный код программы»

Листинг 10. Код работы

```
1 module Budget( Budget(..), concatBudgets, getRemaining) where
2
3 -- Тип Budget отображает бюджет категории: запланированная сумма, потраченная
4   сумма, лимит предупреждения
5 data Budget = Budget{
6   planSum :: Double
7   , spentSum :: Double
8   , limitWarning :: Double
9 } deriving (Eq, Show)
10
11 -- Представитель класса Semigroup для типа Budget
12 -- Операция (< >) объединяет два бюджета: суммирует запланированные и потраченные
13   суммы, выбирает минимальный лимит предупреждения.
14 instance Semigroup Budget where
15 (< >) (Budget p1 s1 l1) (Budget p2 s2 l2) = Budget (p1+p2) (s1+s2) (min
16   l1 l2)
17
18 -- Представитель класса Monoid для типа Budget. Обосновать выбор нейтрального
19   элемента
20 instance Monoid Budget where
21 mempty = Budget 0 0 (1/0)
22 mappend = (< >)
23
24 -- нейтральный элемент - бесконечность, любой бюджет с положительным лимитом его
25   перекрывает"
26 -- Реализовать функцию, которая объединяет бюджеты подкатегорий в общий бюджет
27   категории
28 concatBudgets :: [Budget] -> Budget
29 concatBudgets = mconcat
30
31
32 getRemaining :: Budget -> Double
33 getRemaining budget = planSum budget - spentSum budget
34
35 module TransactionValidator where
36
37 import Transaction
38 import Budget
39
40 -- Определить тип TransactionValidator а как валидатор транзакций с
41   дополнительными данными,
42 -- где Budget - текущий бюджет, Bool - прошла ли транзакция валидацию, а -
43   дополнительная информация
44 newtype TransactionValidator a = TransactionValidator {
45   runValidation :: Transaction -> Budget -> (Bool, a)
46 }
47
48 -- Реализовать представителя класса Functor. Функция fmap преобразует
49   дополнительную информацию валидации
50 instance Functor TransactionValidator where
51 fmap f (TransactionValidator val) = TransactionValidator (\ t b ->
52   let (res, info) = val t b
53   in (res, f info))
```

```

44 -- case val t b of (res, info -> res, f info)
45
46 -- Реализовать функцию, которая модифицирует результаты валидации добавляет(
    рекомендации, логирует решения)
47 simpleValidator :: TransactionValidator String
48 simpleValidator = TransactionValidator $ \t b ->
49 let remaining = getRemaining b
50 transactionSum = summa t
51 isValid = transactionSum <= remaining
52 message = if isValid then "Transaction approved. Remaining after: " ++
    show (remaining - transactionSum)
53 else "Amount " ++ show transactionSum ++ " exceeds remaining budget " ++
    show remaining
54 in (isValid, message)
55
56 -- Реализовать представителя класса Applicative для TransactionValidator a,
    обосновать выбор семантики pure
57 instance Applicative TransactionValidator where
58 pure x = TransactionValidator $ \_ _ -> (True, x)
59 -- по умолчанию всегда True
60 -- (<*>) комбинирует два валидатора; результат проходит только если оба
    валидатора прошли
61 (TransactionValidator f) <*> (TransactionValidator s) =
    TransactionValidator $ \t b ->
62 let (fRes, fFunc) = f t b
63 (vRes, vVal) = s t b
64 in (fRes && vRes, fFunc vVal)
65
66 -- Реализовать функцию, которая комбинирует несколько условий валидации лимиты(,
    категории, временные ограничения)
67 combineValidations :: [TransactionValidator String] ->
    TransactionValidator [String]
68 combineValidations [] = pure []
69 combineValidations (v:vs) = (:) <$> v <*> combineValidations vs
70
71
72 validateAmount :: TransactionValidator String
73 validateAmount = TransactionValidator $ \t b ->
74 let remaining = getRemaining b
75 isValid = summa t <= remaining
76 message = if isValid
77 then "Amount OK: " ++ show (summa t) ++ " <= " ++ show remaining
78 else "Amount exceeded: " ++ show (summa t) ++ " > " ++ show remaining
79 in (isValid, message)
80
81 validateCategory :: TransactionValidator String
82 validateCategory = TransactionValidator $ \t b ->
83 let allowed = [Expense, Income]
84 isValid = transactionType t `elem` allowed
85 message = if isValid
86 then "Category OK: " ++ show (transactionType t)
87 else "Invalid category: " ++ show (transactionType t)
88 in (isValid, message)
89
90 validateWarningLimit :: TransactionValidator String
91 validateWarningLimit = TransactionValidator $ \t b ->

```

```

92 let remainingAfter = getRemaining b - summa t
93 isValid = remainingAfter >= limitWarning b
94 message = if isValid
95 then "Above warning limit: " ++ show remainingAfter ++ " >= " ++ show (
    limitWarning b)
96 else "Below warning limit: " ++ show remainingAfter ++ " < " ++ show (
    limitWarning b)
97 in (isValid, message)
98
99 complexValidator :: TransactionValidator [String]
100 complexValidator = combineValidations [validateAmount, validateCategory,
    validateWarningLimit]
101
102 module Lib
103 ( someFunc,
104   budgetFunc,
105   validationDemo,
106   applicativeFunc
107 ) where
108
109 import Transaction(Transaction(..), TransactionType(..))
110 import TransactionValidator(TransactionValidator(..), simpleValidator,
    complexValidator, combineValidations,
111   validateAmount, validateCategory, validateWarningLimit)
112 import Budget(Budget(..), concatBudgets, getRemaining)
113
114
115
116 someFunc :: IO ()
117 someFunc = putStrLn "someFunc"
118
119 budgetFunc :: IO ()
120 budgetFunc = do
121   let food = Budget 2000 1600 100
122   let transport = Budget 1000 1000 200
123   let dorm = Budget 3000 2700 300
124   let total = concatBudgets [food, transport, dorm]
125   putStrLn $ "Summary budget: " ++ show total
126
127 validationDemo :: IO ()
128 validationDemo = do
129   let testBudget = Budget 5000 2000 500
130   let validTrans = Transaction 1 200.5 Expense "Groceries"
131   let invalidTrans = Transaction 2 4000 Expense "Meat"
132
133   let (result1, msg1) = runValidation simpleValidator validTrans
    testBudget
134   putStrLn $ "Transaction: " ++ show (summa validTrans) ++ " for " ++
    descr validTrans
135   putStrLn $ "Budget: " ++ show (getRemaining testBudget) ++ " in all"
136   putStrLn $ "Result: " ++ show result1
137   putStrLn $ "Message: " ++ msg1
138   putStrLn $ " "
139   let (result2, msg2) = runValidation simpleValidator invalidTrans
    testBudget

```

```

140 putStrLn $ "Transaction: " ++ show (summa invalidTrans) ++ " for " ++
    descr invalidTrans
141 putStrLn $ "Budget: " ++ show (getRemaining testBudget) ++ " in all"
142 putStrLn $ "Result: " ++ show result2
143 putStrLn $ "Message: " ++ msg2
144 putStrLn $ " "
145 let decoratedValidator = fmap (\msg -> ":) " ++ msg ++ " :)")
    simpleValidator
146 let (decResult, decMsg) = runValidation decoratedValidator validTrans
    testBudget
147 putStrLn $ "Decorated: " ++ decMsg
148
149 applicativeFunc :: IO ()
150 applicativeFunc = do
151 let budget = Budget 5000 2000 500
152 let transaction = Transaction 1 800 Expense "Test"
153 let transaction2 = Transaction 2 1900 Transfer "Food"
154
155 let (result, messages) = runValidation (combineValidations [
    validateAmount, validateCategory, validateWarningLimit]) transaction
    budget
156 putStrLn $ "Result: " ++ show result
157 mapM_ putStrLn messages
158 putStrLn $ " "
159 let (res2, messages) = runValidation complexValidator transaction2
    budget
160 putStrLn $ "Result: " ++ show res2
161 mapM_ putStrLn messages

```